Effectful Software Contracts

Anh Khoa Nguyen Fulai Lu

May 5, 2026

Problem

- Existing higher-order contracts can check **values** and **functions**, but they do not systematically handle **effects**.
- Effects include:
  - mutation
  - I/O
  - access control
  - protocol behavior
- Prior work offers many **ad hoc** solutions for specific effects.
- The paper's goal: create one **general contract system** for effectful programs.

Motivation

- In real software, interfaces must specify not only **what a function returns**, but also **what it is allowed to do**.
- Examples:
  - a parallel map may require its argument function to be **pure**
  - a function may promise to mutate **only one specific object**
- Existing contract systems do not express these guarantees well.
- Challenge:
  - contracts should be expressive enough to reason about effects
  - but should not change the behavior of correct programs

Prior Work Comparison

Main takeaway:

- Effect-handler contracts support more cases
- Prior work handles only specific effects

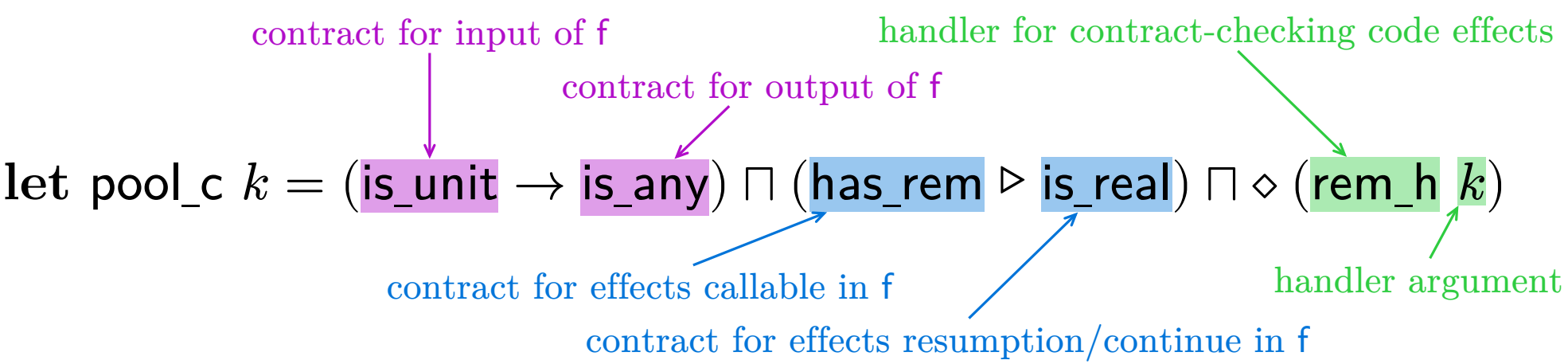
Table 1. Detailed Comparison Matrix

	ALLOW CALL	EXCEPTIONS	FRAMING	GHOST STATE	MUST CALL	NON-REENTRANT	PURE	RESTRICTED EFFECT	TERMINATION	UNION CONTRACTS
Chalin et al. [2006]		•	•	•			•		•	
Tov and Pucella [2010]				•						
Shinnar [2011]		•	•	•			•			
Disney et al. [2011]	•		•	•	•	•				
Keil and Thiemann [2015a]										•
Scholliers et al. [2015]	•			•	•					
Moore et al. [2016]	•			•	•	•				
Dimoulas et al. [2016]	•			•	•					
Bañados Schwerter [2016]		•					•	•		
Williams et al. [2018]										•
Nguyễn et al. [2019]									•	
Moy and Felleisen [2023]	•		•	•	•	•				
Effect-Handler Contracts	•	•	•	•	•	•	•	•	•	•

Key Idea

- The paper combines **software contracts** with **effect handlers**.
- Effects are modeled as **effect requests** handled at runtime.
- Contracts can then constrain:
  - which effects a function may perform
  - how those effects may resume
- Main design idea: **two levels of effects**
  - **main effects** for ordinary program execution
  - **contract effects** for contract checking
- This separation prevents contracts from interfering with the main program.

Summary



let f : pool_c some_k = ...

Solution Details

- The paper introduces two important contract forms:
  - contracts that restrict the effects of a protected function
  - contracts that let contract code install its own handlers
- This allows contracts to:
  - check effects
  - perform effectful monitoring
  - use state or other effects internally
- Key goal: preserve **erasure**
  - on correct programs, contracts should not change results
  - except to signal contract violations

Questions?

Thanks for listening.